



Successfully Built Microservices with Eureka + Load Balancing

Today I completed a production-style **Microservices setup using Eureka Service Registry & Client-Side Load Balancer**.

Instead of services calling each other using **hardcoded URLs**, the system now discovers services dynamically — making it **scalable, fault-tolerant and cloud-ready**.



System Architecture

Service	Responsibility	Port
Eureka Server	Registers and tracks running microservices	8761
Student-Service (Producer)	Provides student details	9091 / 9092 / 9093
Result-Service (Consumer)	Fetches Student-Service details via Eureka	8080



Why was Eureka needed?

Traditional REST communication breaks when:

- ✗ Service port changes
- ✗ Multiple service instances run
- ✗ One service shuts down unexpectedly



Eureka solves all of these

- Every service **registers itself**
- Other services **ask Eureka for available running instances**

- Load balancer distributes traffic **automatically across instances**

Request Flow

Client → Result-Service → Eureka → Student-Service (Instance 1 / 2 / N) → Result-Service → Client

No hardcoded URLs.

Service discovery happens **at runtime** 🔥

Load Balancer Test Result

I launched **two instances** of Student-Service on ports `9091` and `9092`.

When refreshing `/result/info`, the response alternated like:

```
portNo: 9091 ← served by instance-1
portNo: 9092 ← served by instance-2
```

This proves **Round-Robin load balancing is working**.

Key Takeaways

- ✓ Microservices must be **loosely coupled**
- ✓ Never hardcode service URLs — always use a **service registry**
- ✓ Distributed systems require **dynamic service lookup**
- ✓ Load balancer allows **horizontal scaling**
- ✓ Eureka provides **visibility, health status & dynamic discovery**

Final Thought

This project helped me move from building “normal REST APIs” to **thinking like a real distributed systems engineer** 🚀

Next goal → **Integrate Spring Cloud API Gateway + Feign Client + Config Server**

 **COMPLETE PROJECT CODE —>**

```
1 //Eureka Server -
2 package com.shivraj;
3
4 import org.springframework.boot.SpringApplication;
5 import org.springframework.boot.autoconfigure.SpringBootApplication;
6 import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
7
8 @SpringBootApplication
9 @EnableEurekaServer
10 public class Sbms02EurekaServerApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(Sbms02EurekaServerApplication.class, args);
14     }
15
16 }
17
```

```
1 #Result Server - properties file
2 spring.application.name=SBMS02-EurekaServer
3
4 server.port=8761
5
6 eureka.client.register-with-eureka=false
7 eureka.client.fetch-registry=false
8
```

```
1 //Student Server -
2 package com.shivraj;
3
4 import org.springframework.boot.SpringApplication;
5 import org.springframework.boot.autoconfigure.SpringBootApplication;
6 import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
7
8 @SpringBootApplication
9 @EnableDiscoveryClient
10 public class Sbms02StudentServiceApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(Sbms02StudentServiceApplication.class, args);
14     }
15
16 }
17
```

```
1 //Student Server - controller
2 package com.shivraj.controller;
3
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RestController;
10
11 import com.shivraj.model.Student;
12
13 @RestController
14 @RequestMapping("/student-api")
15 public class StudentController {
16
17     @Value("${server.port}")
18     private int port;
19
20     @Value("${eureka.instance.instance-id}")
21     private String instanceId;
22
23     @GetMapping("/studentInfo")
24     public ResponseEntity<Student> getStudentDetail() {
25         Student student = new Student(101, "Shivraj Gawande", 95.0, port,
instanceId);
26         return new ResponseEntity<>(student, HttpStatus.OK);
27     }
28 }
29
30
```

```
1 //Student Server - proerties file
2
3 server.port=9091
4 spring.application.name=Student-Service
5 eureka.client.service-url.default-zone=http://localhost:8761/eureka
6 eureka.instance.instance-id=${spring.application.name}:${random.value}
7
```



```
1 //Result Server -
2
3 package com.shivraj;
4
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7 import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
8
9 @SpringBootApplication
10 @EnableDiscoveryClient
11 public class Sbms02ResultServiceApplication {
12
13     public static void main(String[] args) {
14         SpringApplication.run(Sbms02ResultServiceApplication.class, args);
15     }
16
17 }
18
```

```
1 //Result Server - client
2
3 package com.shivraj.client;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.cloud.client.ServiceInstance;
7 import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.stereotype.Component;
10 import org.springframework.web.client.RestTemplate;
11
12 @Component
13 public class StudentServiceConsumerClient {
14
15     @Autowired
16     private LoadBalancerClient client;
17
18     public String getStudentInfo() {
19         ServiceInstance serviceInstance = client.choose("Student-Service");
20
21         String url = serviceInstance.getUri().toString()+"/student-
api/studentInfo";
22
23         RestTemplate template = new RestTemplate();
24
25         ResponseEntity<String> responseEntity = template.getForEntity(url,
String.class);
26         String response = responseEntity.getBody();
27
28         return response;
29     }
30
31 }
32
```

```
1 //Result Server - controller
2 package com.shivraj.controller;
3
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.RequestMapping;
9 import org.springframework.web.bind.annotation.RestController;
10
11 import com.shivraj.client.StudentServiceConsumerClient;
12
13 @RestController
14 @RequestMapping("/result")
15 public class ResultController {
16
17     @Autowired
18     private StudentServiceConsumerClient client;
19
20     @GetMapping("/info")
21     public ResponseEntity<String> getStudentResult(){
22
23         String response = "Student Details Along with result are listed below :
24 \n"+client.getStudentInfo();
25
26         return new ResponseEntity<>(response, HttpStatus.OK);
27     }
28 }
29
30
```

```
1 //Result Server - properties file
2
3 server.port=8080
4 spring.application.name=Result-Service
5 eureka.client.service-url.default-zone=http://localhost:8761/eureka
6
7
8
9
```

```
1  //Student - Model class
2
3  package com.shivraj.model;
4
5  import lombok.AllArgsConstructor;
6  import lombok.Data;
7  import lombok.NoArgsConstructor;
8
9
10 @Data
11 @AllArgsConstructor
12 @NoArgsConstructor
13 public class Student {
14
15     private int roll;
16     private String name;
17     private double percentage;
18     private int portNo;
19     private String instanceId;
20
21 }
22
23
```